Containerization and DevOps

Assignment 1

Submitted to Dr. Prateek Raj Gautam

Name: Arnav Sharma
Batch: 2
SAPID: 500120678

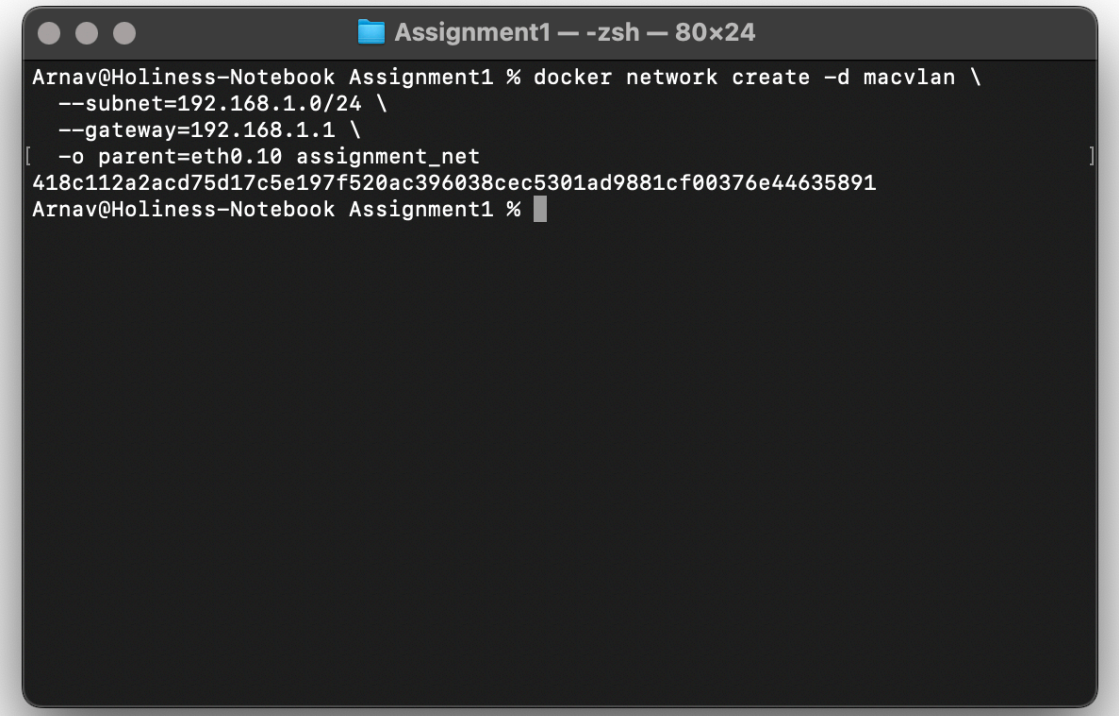
Objective:

The goal of this project was to design and deploy a containerized full-stack application consisting of a Node.js/Express backend and a PostgreSQL database. The deployment emphasizes production-ready standards, including image optimization, non-root security, and advanced network stack orchestration using Docker Compose and Macvlan.

Network Topology:

Implemented a flat network topology using the **Macvlan** driver. Unlike the default bridge network, Macvlan allows containers to appear as physical devices on the local network, each with its own unique MAC address and IP assigned from the host's subnet.

- **Backend IP:** 192.168.1.100
- **Database IP:** 192.168.1.101
- **Gateway:** 192.168.1.1



Separate Dockerfiles

Two Dockerfiles implemented for both backend and database and used a docker-compose.yml to build both in totality

backend — vim Dockerfile — 80×24

```
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

# Stage 2: Runtime
FROM node:20-alpine
WORKDIR /app
# Create non-root user for security
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=builder /app/node_modules ./node_modules
COPY . .
USER appuser
EXPOSE 3000
CMD ["node", "index.js"]
~
~
~
~
~
~
~
-- INSERT --                                        15,12         All
```

database — vim Dockerfile — 80×24

```
FROM postgres:15-alpine
LABEL type="assignment-db"
# Standard postgres alpine image is already optimized
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
-- INSERT --                                         3,54         All
```

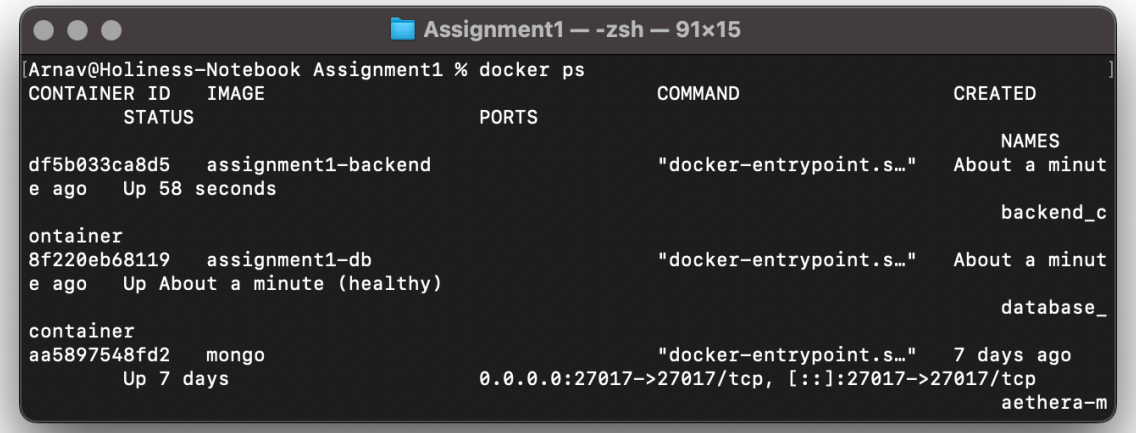
Build Optimisation:

Utilized **Docker Multi-stage builds** to separate the "Build-time" environment from the "Runtime" environment.

- **Stage 1 (Builder):** Uses a full `Node.js` image to install dependencies and compile assets.
- **Stage 2 (Runner):** Uses a minimal `alpine` image. We only copy the `node_modules` and source code from the builder.

Image Strategy	Base Image	Size (Approx)
Standard Build	node:latest	1.1 GB
Optimised Build	node:20-alpine	180 MB

Reducing the image size decreases the attack surface, speeds up CI/CD pipeline deployments, and reduces storage costs in a production registry.



Security and Configuration

In both the Backend and Database Dockerfiles, we explicitly avoided running processes as root.

- **Implementation:** Used `addgroup -S appgroup && adduser -S appuser -G appgroup`.
- **Reasoning:** If a vulnerability is exploited in the Express app, the attacker only gains the limited permissions of the `appuser`, preventing them from modifying the host system or container kernel.

Environment Variable Management

Sensitive credentials (DB passwords, Usernames) were kept out of the source code and the Dockerfile. Instead, we used a `.env` file passed into the containers via Docker Compose.

Macvlan vs Ipvlan:

During the design phase, we compared Macvlan and Ipvlan:

- **Macvlan:** Each container gets a unique MAC. This is the most "transparent" method but can be restricted by network switches that limit MAC addresses per port (Port Security).
- **Ipvlan (L2):** Containers share the host's MAC address. This is more efficient for high-density environments and bypasses MAC limits on switches.

The Host Isolation Issue

A critical discovery during testing was that the **Host cannot ping the Containers** over a Macvlan network.

- **Cause:** The Linux kernel's networking stack prevents traffic from being looped back from a virtual interface to the physical interface for security reasons.
- **Solution:** In a real-world production server, a separate "bridge" or "dummy" interface would be created on the host to route traffic internally to the Macvlan subnet.

Data Persistence

To ensure zero data loss during container restarts or updates, we implemented a **Named Volume** (`pg_data`) for PostgreSQL.

- **Validation:** Running `docker-compose down` followed by `docker-compose up` confirmed that the table records created in previous sessions remained intact.

Health Checks

We implemented a custom health check for the database using `pg_isready`. The backend service was configured with `depends_on: condition: service_healthy`, ensuring the API only starts once the database is fully ready to accept connections.